

and music (unless created by you) will be streamed to you on-demand at a flat fee, hopefully putting an end to digital piracy. We can almost hear the MPAA and RIAA heave a sigh of relief. Of course, in the midst of all this, we're assuming there's ample bandwidth available.

...But down to earth

But don't fret yet, we're not completely abandoning the concept of locally stored files. We understand that you need important files and folders available to you at all times, in the event of a network outage. However, we'd like to change the way we store files locally. Instead of the current folder-based hierarchy which requires you to memorize the location of your files (not to mention the number of clicks it takes to get to a deeply nested folder), we'd like to adopt a flat-hierarchy, where your files can be retrieved with just a few keystrokes. Maybe even attach tags to your files to aid faster search. This search engine will integrate data from both online and offline sources, similar to Ubuntu's Unity Dash, thus blurring the line between locally stored content and online data. So say you're searching for a particular video; instead of navigating to your Videos folder and traversing a bunch of subfolders, you can just hit a key to bring up the search box, start typing into it, and watch the results intelligently appear from among those on YouTube, Vimeo and other video sharing sites, as well as those stored locally. So if you're searching for a video of a cat, the engine will be able to process the contents of a video, determine that the video features a cat, and display it in the results, even if the actual filename doesn't contain the word 'cat'. Such algorithms exist in a nascent state, examples being Soundhound or Shazam that fingerprint songs, and identify sound clips by comparing this information against a database of known songs.

Happy to help

Another feature we'd like is an OS that doesn't require any maintenance on part of the user, and one that shuts down and boots up instantly. Current OSes can take several minutes to boot up completely, as they require the loading of several daemons, processes or services to manage the various aspects of the system. Another shortcoming of current platforms is the cruft that grows after having used a system for a while, with leftover program files, invalid links, registry

entries or unnecessary services running at boot up. This inevitably requires the (power) user to reinstall the OS almost every year. In an OS that is, at its core, a web browser, not only will leftover files not be an issue, but frequently accessed apps and websites can be cached in memory for faster access. This will mean that performance will actually increase over time as the device learns your browsing habits.

The task of updating the software and applications isn't something the user will have to worry about, since it will be taken care of the server-side. But we would like to see a built-in, system-wide version-control for all changes that a user makes. This will record even the most minute changes made to files and settings, allowing the user to rollback the changes to any previous configuration with ease. This will also erase the archaic practice of having to "save" one's work periodically to prevent data loss, since all changes will be recorded asynchronously, similar to Google Docs. While it may sound lofty have a look at the Zetabyte Filesystem (ZFS) on the Solaris platform. Among other features, ZFS supports automatic and incremental 'snapshots' which are read-only copies of the filesystem's state at a point in time. It allows you to revert to a previous snapshot in case of data loss or corruption, and has a live Disk-Scrubber, which checks for and repairs corrupted blocks. Oh, and we'd also like apps to be able to share

data between each other, just imagine the possibilities with that! Throw in a fair bit of AI and Siri will look like a down syndrome dodo. (Not that Siri is any Pepper Potts clone today, anyway).

Open and shut case

From a developer's perspective, one of the most fundamental questions with software is whether it is Open Source or Closed (Proprietary). This single factor can go a long way in influencing developers to embrace the platform. Open Source software tends to appeal more to developers, as it gives them a lot more freedom to tinker around with the internals of the system. Closed systems don't offer this, and the amount of freedom available is left to the OEM. However, opening up the internals of the system can pose serious threats, exposing the system to malware infections. Furthermore, giving access to system critical files to the layman can pose risks – such a user may accidentally modify some part of the system, rendering it unus-



Beam me data up scotty

A RECENT REPORT BY GOLDMAN SACHS STATES:

Google's Android owns the lion's share of the consumer device market at

42%

Microsoft Windows and Windows Phone at

Apple's iOS and OS X (combined) at

24%

20%

This is in stark contrast to the landscape just five years ago, when Microsoft commanded **42%** of the total market, with Apple at **21%** and Google's Android nowhere in sight. This just goes to show the breakneck speed at which the computing industry has been innovating and reinventing itself these past couple of years. (<http://dgit.in/VQnC4E>)

able. Our ideal Operating System would aim to have the flexibility of Open Source, but with the ease of use offered by a closed one. This could be implemented by means of a software 'switch' that would be off by default, but can be switched 'on' to give access to the entire system. The layperson wouldn't ever have the need to use this switch, and it would be assumed that a developer wanting to unlock a device would be aware of the potential risks.

Our ideal Operating System will enable low-power, portable devices to become ubiquitous, and achieve far greater functionality than today. It will also give users a great deal of freedom and choice in the software they use, instead of tying them into one particular set of services / devices, as open standards compliant hardware and software will replace the current wave of proprietary and restrictive protocols. **d**